

SCT Final TP - Microservices

Group Members:

1. Jonathan Aussant
2. Abdullah Shahid

Demo Video: <https://www.loom.com/share/a2e96f053e4642f4bbf0620445ef047c>

Cloud Platform Choice and Initial Architecture

We decided to deploy the project on AWS because we already had access to a Free Tier account. Our first idea was to use Amazon ECS, since it aligns well with container-based architectures. But after reviewing the *microservices-demo* repository, we noticed that the project already provides ready-to-use Docker Compose files under `deploy/docker-compose`.

Since the existing setup was optimized for Docker Compose rather than container orchestration, we chose a simpler approach: running everything on EC2. We provisioned two Ubuntu-based t3.small instances. One instance hosted the full Sock Shop application and its microservices, while the second was dedicated to running the load-testing service. Both machines were created with the same security group and authenticated through a single PEM key pair to keep management simple.

EC2 Setup: Sock Shop Deployment

After creating the EC2 instance for the Sock Shop application, We assigned an **Elastic IP** to ensure a stable public endpoint. We connected to the machine using the PEM key:

```
ssh -i "jaas.pem"  
ubuntu@ec2-15-236-88-3.eu-west-3.compute.amazonaws.com
```

Inside the instance, we worked under `/opt` and installed the required tools:

```
sudo apt update  
sudo apt install -y docker.io docker-compose git  
sudo usermod -aG docker ubuntu
```

After re-logging to apply Docker permissions, we downloaded the *microservices-demo* source code:

```
git clone https://github.com/microservices-demo/microservices-demo.git
```

```
cd microservices-demo/deploy/docker
```

we attempted to bring the stack online using:

```
docker-compose up -d
```

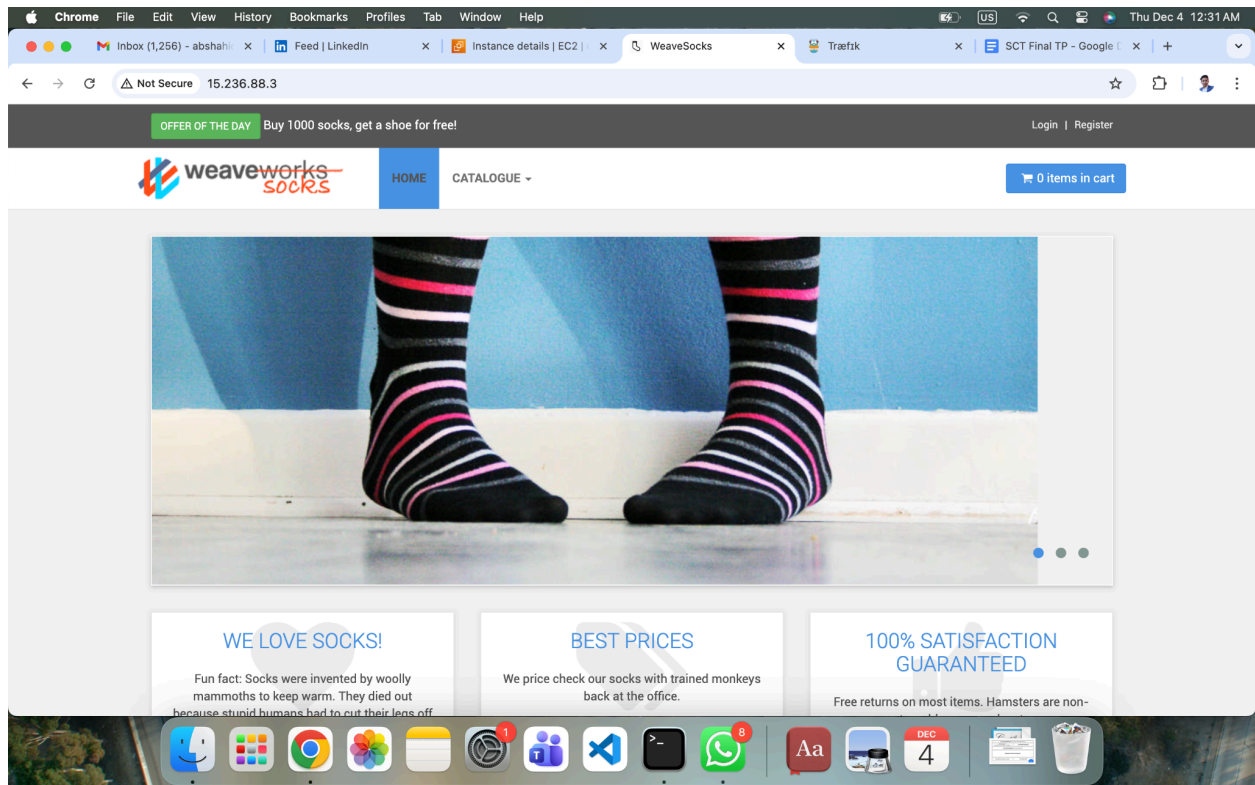
At this point we ran into a performance issue. The initial t3.small instance repeatedly became unresponsive while the containers were starting. The root cause was insufficient CPU and memory to support all the Sock Shop microservices. Since the Free Tier didn't allow larger t-class instances, we switched to a **c7i-flex.large**, which provided enough resources for Docker Compose to start the full stack reliably.

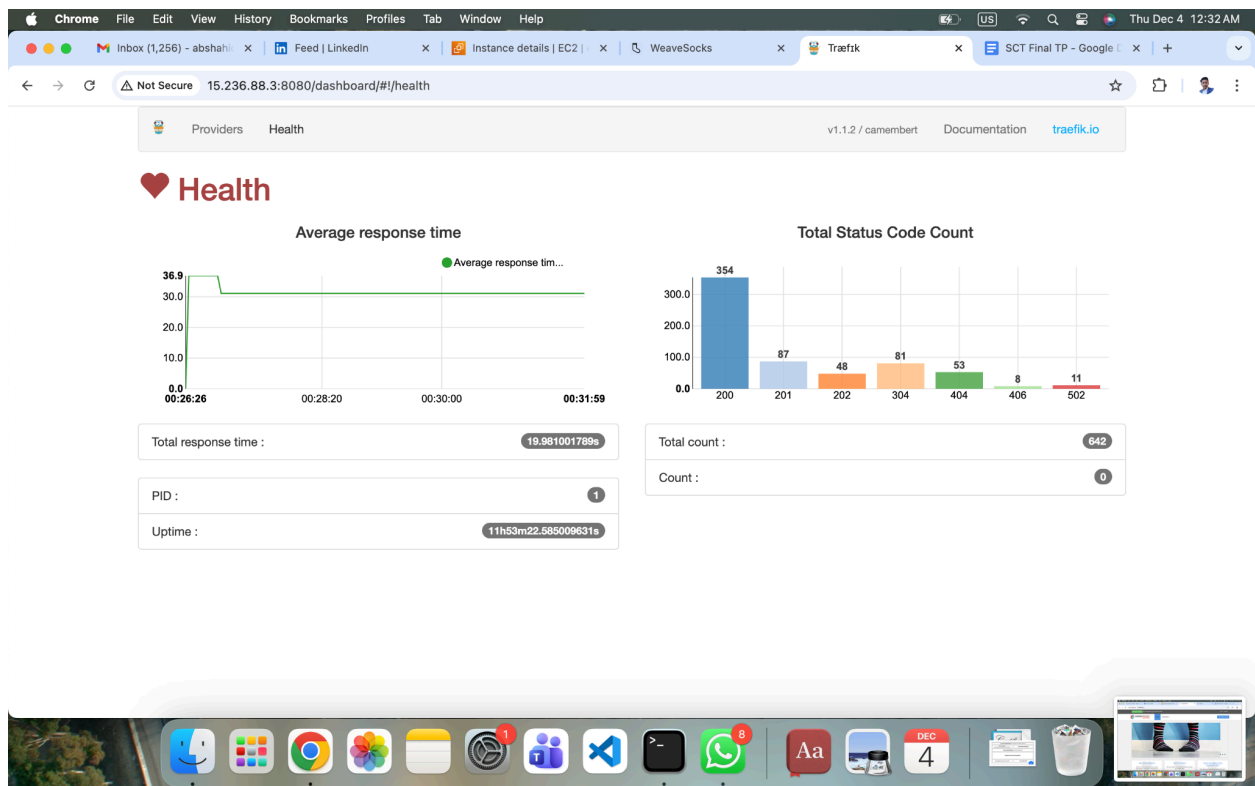
After the upgrade, the deployment completed successfully.

To make the UI accessible externally, we updated the Security Group to allow:

- **Port 80** for the Sock Shop frontend
- **Port 8080** for Traefik

We were able to open the application from a browser by using the Elastic IP address and corresponding ports.





EC2 Setup for Load Testing

We created a second EC2 instance dedicated to workload generation. During setup, we selected the same PEM key pair as the main instance, chose Ubuntu as the OS, and opened port 22 for SSH.

After the instance launched, we connected to it and installed Docker and Git:

```
sudo apt update
sudo apt install -y docker.io git
sudo usermod -aG docker ubuntu
```

We then cloned the load-test repository:

```
git clone https://github.com/microservices-demo/load-test.git
cd load-test
```

Building the Load-Test Image

We attempted to build the Docker image:

```
sudo docker build -t sockshop-loadtest .
```

The build failed with the error:

```
locustio requires Python '>= 3.6' but the running Python is 2.7.16
```

The Dockerfile used `python:2.7-wheezy`, which did not match Locust's requirements. We forked the repository to our own GitHub and changed the first line to use Python 3.9:

```
FROM python:3.9
```

The build continued to fail, so on the **teacher's recommendation** we switched to the pre-built image provided in the second repository.

Switch to Built-In Load Test (user-sim)

After the issues with the separate load-test instance, we shut it down and decided to use the built-in load generator included in the main microservices-demo repository.

Inside `microservices-demo/deploy/docker-compose/docker-compose.yml`, we found the predefined service:

```
user-sim:
  image: weaveworksdemos/load-test:0.1.1
  cap_drop:
    - all
  read_only: true
  hostname: user-simulator
  command: "-d 60 -r 200 -c 2 -h edge-router"
```

To launch the load test, we ran:

```
sudo docker-compose up user-sim
```

This triggered the workload generation and the terminal displayed the output produced by the load tester

```

Attaching to docker-compose_user-sim_1
user-sim_1 | Locust file: /config/locustfile.py
user-sim_1 | Will run /config/locustfile.py against edge-router. Spawning 2 clients and 200 total requests.
user-sim_1 | [2025-12-03 23:26:54,057] user-simulator/INFO/locust.main: Starting Locust @ 7.5
user-sim_1 | [2025-12-03 23:26:54,057] user-simulator/INFO/locust.runners: Hatching and spawning 2 clients at the rate 5 clients/s...
user-sim_1 | [2025-12-03 23:26:54,460] user-simulator/INFO/locust.runners: All locusts hatched: Web: 2
user-sim_1 | [2025-12-03 23:26:54,460] user-simulator/INFO/locust.runners: Resetting stats
user-sim_1 |
user-sim_1 | [2025-12-03 23:26:56,855] user-simulator/INFO/locust.runners: All locusts dead
user-sim_1 |
user-sim_1 | [2025-12-03 23:26:56,855] user-simulator/INFO/locust.main: Shutting down (exit code 1), bye.
user-sim_1 |
user-sim_1 | Name      # reqs      # falls    Avg    Min    Max    Median    req/s
user-sim_1 | -----
user-sim_1 | GET /      22      0(0.00%)    5      2     44      3    0.00
user-sim_1 | GET /basket.html  22      0(0.00%)    6      3     44      4    0.00
user-sim_1 | DELETE /cart  22      0(0.00%)   10      7     14     10    0.00
user-sim_1 | POST /cart  22      0(0.00%)   20     14     29     21    0.00
user-sim_1 | GET /catalogue  23      0(0.00%)    6      5     13      6    0.00
user-sim_1 | GET /category.html  22      0(0.00%)    3      2     10      3    0.00
user-sim_1 | GET /detail.html?id=03f6f6ac-1896-4ceb-bd69-b798f85c6e0b  3      0(0.00%)    3      3      4      3    0.00
user-sim_1 | GET /detail.html?id=3395a43e-2d88-40de-b95f-e00e1502085b  4      0(0.00%)    5      3     10      3    0.00
user-sim_1 | GET /detail.html?id=510a0d7e-8e83-4193-b483-e27e09ddc34d  2      0(0.00%)    3      3      4      3    0.00
user-sim_1 | GET /detail.html?id=808a2de1-1aa0-4c25-a909-6612e8f29a38  3      0(0.00%)    3      3      4      3    0.00
user-sim_1 | GET /detail.html?id=819e1fbf-8b7e-4fed-811f-693534916a8b  4      0(0.00%)    3      3      4      3    0.00
user-sim_1 | GET /detail.html?id=837db141-399e-4c1f-9abc-bace4029b0ac  1      0(0.00%)    3      3      3      3    0.00
user-sim_1 | GET /detail.html?id=a0a4f044-b040-410d-8ead-4de0446a0c7e  3      0(0.00%)    4      3      5      4    0.00
user-sim_1 | GET /detail.html?id=d3588630-ad8e-49df-bbd7-31677efb246  1      0(0.00%)    3      3      3      3    0.00
user-sim_1 | GET /detail.html?id=zz24f044-b040-410d-8ead-4de0446a0c7e  1      0(0.00%)    2      2      2      2    0.00
user-sim_1 | GET /login  22      0(0.00%)   55     51     76     54    0.00
user-sim_1 | POST /orders  19      4(17.39%)  102     76    138     95    0.00
user-sim_1 |
user-sim_1 | Total      196      4(2.04%)
user-sim_1 |
user-sim_1 | Percentage of the requests completed within given times
user-sim_1 | Name      # reqs    50%    60%    75%    80%    90%    95%    98%    99%   100%
user-sim_1 | -----
user-sim_1 | GET /      22      3      3      4      5      6     11     44     44     44
user-sim_1 | GET /basket.html  22      4      5      6      6      7      7     44     44     44
user-sim_1 | DELETE /cart  22     10     11     13     13     14     14     14     14     14
user-sim_1 | POST /cart  22     21     23     24     24     28     28     29     29     29
user-sim_1 | GET /catalogue  23      6      6      6      7      7      7     13     13     13
user-sim_1 | GET /category.html  22      4      4      5      5      5      5     10     10     10
user-sim_1 | GET /detail.html?id=03f6f6ac-1896-4ceb-bd69-b798f85c6e0b  3      3      3      4      4      4      4      4      4      4
user-sim_1 | GET /detail.html?id=3395a43e-2d88-40de-b95f-e00e1502085b  4      4      4      10     10     10     10     10     10     10
user-sim_1 | GET /detail.html?id=510a0d7e-8e83-4193-b483-e27e09ddc34d  2      4      4      4      4      4      4      4      4      4
user-sim_1 | GET /detail.html?id=808a2de1-1aa0-4c25-a909-6612e8f29a38  3      3      3      4      4      4      4      4      4      4
user-sim_1 | GET /detail.html?id=819e1fbf-8b7e-4fed-811f-693534916a8b  4      3      3      4      4      4      4      4      4      4
user-sim_1 | GET /detail.html?id=837db141-399e-4c1f-9abc-bace4029b0ac  1      3      3      3      3      3      3      3      3      3
user-sim_1 | GET /detail.html?id=a0a4f044-b040-410d-8ead-4de0446a0c7e  3      4      4      5      5      5      5      5      5      5
user-sim_1 | GET /detail.html?id=d3588630-ad8e-49df-bbd7-31677efb246  1      3      3      3      3      3      3      3      3      3
user-sim_1 | GET /detail.html?id=zz24f044-b040-410d-8ead-4de0446a0c7e  1      2      2      2      2      2      2      2      2      2
user-sim_1 | GET /login  22     55     55     55     57     58     61     76     76     76
user-sim_1 | POST /orders  19     95     100    120    130    130    140    140    140    138
user-sim_1 |
user-sim_1 |
user-sim_1 | Error report
user-sim_1 | # occurrences    Error
user-sim_1 | -----
user-sim_1 | 4      POST /orders: "HTTPError(u'406 Client Error: Not Acceptable for url: http://edge-router/orders'.)"
user-sim_1 |
user-sim_1 |
user-sim_1 | done
docker-compose_user-sim_1 exited with code 0

```

Automated Load Testing

To meet the requirement of automated load generation, we used the `user-sim` service already defined in the `docker-compose.yml`. This service automatically produces traffic against the application through the `edge-router`.

We started it with:

```
sudo docker-compose up -d user-sim
```

This launched the `weaveworksdemos/load-test` container in detached mode, which continuously generated requests and stressed the Sock Shop application without manual intervention.

Logging and Monitoring Setup

EC2 provides basic monitoring out of the box. Metrics like CPUUtilization, NetworkIn, NetworkOut, DiskReadOps, and DiskWriteOps were already visible in CloudWatch under **Metrics** → **EC2**.

To add more advanced monitoring, we explored the Docker Compose setup and found the file `docker-compose.monitoring.yml`, which includes Grafana, Prometheus, and Alertmanager.

We started the monitoring stack with:

```
sudo docker-compose -f docker-compose.monitoring.yml up -d
```

```
Last login: Thu Dec 4 06:47:34 on console
macmucci-MacBook-Pro - % cd Downloads
macmucci-MacBook-Pro Downloads % ssh -i "joas.pem" ubuntu@ec2-15-236-88-3.eu-west-3.compute.amazonaws.com
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1015-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Thu Dec 4 06:35:49 UTC 2025

System load:  0.45          Temperature:    -273.1 C
Usage of /:   8.0% of 95.82GB Processes:      215
Memory usage: 65%          Users logged in: 0
Swap usage:  0%            IPv4 address for enp39s0: 172.31.9.218

Expanded Security Maintenance for Applications is not enabled.

9 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

*** System restart required ***
Last login: Wed Dec 3 23:21:16 2025 from 46.193.64.32
ubuntu@ip-172-31-9-218:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED    STATUS    PORTS                               NAMES
f8647d1e415f   grafana/grafana                    "/run.sh"               6 hours ago Up 6 hours    0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp    docker-compose_grafana_1
68dc02c2724e   prom/prometheus                    "/bin/prometheus --c..." 6 hours ago Up 6 hours    0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp    prometheus
95d892f3f23d   prom/alertmanager                  "/bin/alertmanager -..." 6 hours ago Up 6 hours    0.0.0.0:9093->9093/tcp, [::]:9093->9093/tcp    docker-compose_alertmanager_1
fc6b92b71c83   quay.io/prometheus/node-exporter:v1.1.2 "/bin/node_exporter ..." 6 hours ago Up 6 hours    0.0.0.0:9100->9100/tcp, [::]:9100->9100/tcp    nodeexporter
96438ee19b05   weaveworksdemos/payment:0.4.3     "/app --port=80"        19 hours ago Up 19 hours    80/tcp    docker-compose_payment_1
db0ad817c7d6   weaveworksdemos/orders:0.4.7      "/usr/local/bin/java..." 19 hours ago Up 19 hours    80/tcp    docker-compose_orders_1
94dc1190417   weaveworksdemos/catalogue:0.3.5   "/app --port=80"        19 hours ago Up 19 hours    80/tcp    docker-compose_catalogue_1
cf4dd88b2e0b   weaveworksdemos/queue-master:0.3.1 "/usr/local/bin/java..." 19 hours ago Up 19 hours    3386/tcp  docker-compose_queue-master_1
b47bd305c395   weaveworksdemos/catalogue-db:0.3.0 "docker-entrypoint.s..." 19 hours ago Up 19 hours    4369/tcp, 5671-5672/tcp, 25672/tcp  docker-compose_catalogue-db_1
409b34f62a2e   rabbitmq:3.6.8                    "docker-entrypoint.s..." 19 hours ago Up 19 hours    8079/tcp  docker-compose_rabbitmq_1
8a77a05d568b   weaveworksdemos/front-end:0.3.12   "/usr/local/bin/npm ..." 19 hours ago Up 19 hours    27017/tcp  docker-compose_front-end_1
475a4ff728d5   mongo:3.4                          "docker-entrypoint.s..." 19 hours ago Up 19 hours    27017/tcp  docker-compose_mongo_1
a7ef6c697c70   weaveworksdemos/edge-router:0.1.1 "traefik"               19 hours ago Up 19 hours    0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp  docker-compose_edge-router_1
883c6af72549   weaveworksdemos/user-db:0.4.0      "/entrypoint.sh mong..." 19 hours ago Up 19 hours    27017/tcp  docker-compose_user-db_1
6a766830d0e0   weaveworksdemos/carts:0.4.6        "/usr/local/bin/java..." 19 hours ago Up 19 hours    80/tcp    docker-compose_carts_1
243da33d0c77   weaveworksdemos/user:0.4.4         "/user --port=80"        19 hours ago Up 19 hours    80/tcp    docker-compose_user_1
4f6a1a8d47d7   weaveworksdemos/shipping:0.4.8     "/usr/local/bin/java..." 19 hours ago Up 19 hours    27017/tcp  docker-compose_shipping_1
4db2f4bc9bea   mongo:3.4                          "docker-entrypoint.s..." 19 hours ago Up 19 hours    27017/tcp  docker-compose_orders-db_1
ubuntu@ip-172-31-9-218:~$
```

We then exposed the necessary ports in the security group:

- **3000** for Grafana
- **9090** for Prometheus
- **9093** for Alertmanager

This allowed us to access the full monitoring suite from the browser using the Elastic IP.

Prometheus Verification

After starting the monitoring stack, we opened Prometheus at:

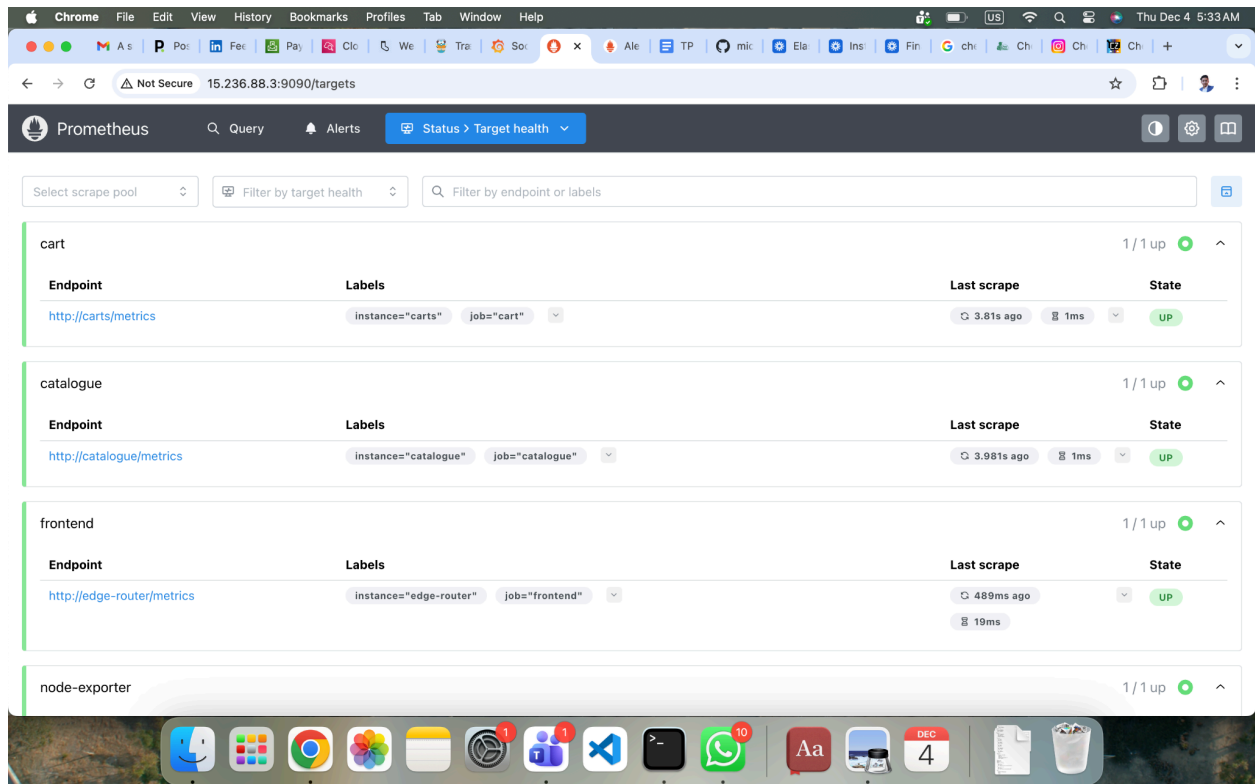
<http://15.236.88.3:9090/>

In **Status** → **Targets**, we could see all expected jobs, including:

- `sock-shop-*` services (carts, front-end, edge-router, etc.)
- `nodeexporter`
- other microservices from the stack

All targets showed the **UP** state.

We also tested PromQL by running the `up` query, and the results confirmed that Prometheus was successfully scraping metrics from every service.



Grafana Dashboard Setup

We then opened Grafana on port 3000 and logged in with the default credentials:

- **Username:** admin
- **Password:** foobar

We first configured Prometheus as the data source inside Grafana so the dashboards could query the metrics collected from our services. After confirming that the basic `up` panel worked,

we added the required monitoring panels. Using the Prometheus data source, we created queries such as:

QPS

```
sum(rate(http_server_requests_seconds_count[1m]))
```

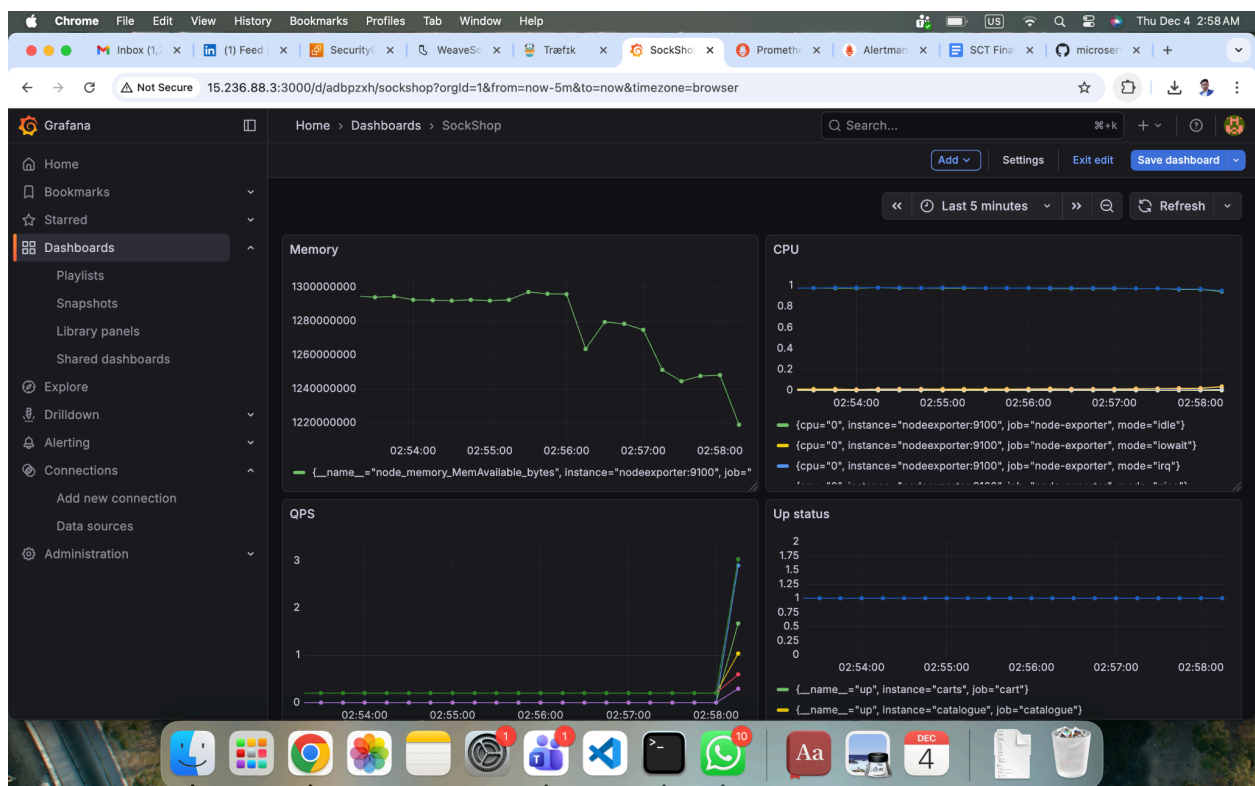
CPU Usage

```
rate(node_cpu_seconds_total[1m])
```

Memory Availability

```
node_memory_MemAvailable_bytes
```

We used these panels to build a dashboard showing request load, CPU activity, and memory usage while the application was under stress.



Alert Manager

AlertmanagerAlertsSilencesStatusSettingsHelp

New Silence

Status

Uptime:

2025-12-04T00:52:34.737Z

Cluster Status

Name:

01KBKDHESER27EFFETES3H1M17

Status:

ready

Peers:

Name: 01KBKDHESER27EFFETES3H1M17

Address: 172.18.0.17:9094

Version Information

Branch:

HEAD

BuildDate:

20251104-13:09:23

BuildUser:

root@f4d6cb29d2f5

GoVersion:

go1.25.3

Revision:

2f0cff51fd1cc761eeb671db43736341ca2ab511

Version:

0.29.0

CloudWatch

eu-west-3.console.aws.amazon.com/cloudwatch/home?region=eu-west-3#home:dashboards/EC2

Account ID: 1229-0210-1837Abdullah%20Shahid

CloudWatch > EC2

Cross account unavailable

1h3h12h1d1wCustomUTC timezone

ActionsExplore relatedAdd to dashboard

EC2

Filter by resource group

Alarm recommendations

CPU Utilization: Average

Percent

6.36

4.18

1.99

00:0001:0002:00

I-0c8795ea49668dfef (Socksopapp)

I-09ed93e41b9f19501 (WorkLoad)

DiskReadBytes: Average

No unit

No data available.

Try adjusting the dashboard time range.

00:0001:0002:00

I-0c8795ea49668dfef (Socksopapp)

I-09ed93e41b9f19501 (WorkLoad)

DiskReadOps: Average

No unit

No data available.

Try adjusting the dashboard time range.

00:0001:0002:00

I-0c8795ea49668dfef (Socksopapp)

I-09ed93e41b9f19501 (WorkLoad)

DiskWriteBytes: Average

No unit

No data available.

Try adjusting the dashboard time range.

00:0001:0002:00

I-0c8795ea49668dfef (Socksopapp)

I-09ed93e41b9f19501 (WorkLoad)

DiskWriteOps: Average

No unit

No data available.

Try adjusting the dashboard time range.

00:0001:0002:00

I-0c8795ea49668dfef (Socksopapp)

I-09ed93e41b9f19501 (WorkLoad)

NetworkIn: Average

Bytes

89.6M

44.8M

18

00:0001:0002:00

I-0c8795ea49668dfef (Socksopapp)

I-09ed93e41b9f19501 (WorkLoad)

CloudShellFeedbackConsole Mobile App

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy

Elasticity Support

We added elasticity support by identifying the most stressed services through our Grafana dashboards. Edge-router, Orders, and User showed the highest load, with increased QPS, growing latency, and noticeable CPU spikes during the test.

To scale these services, we used Docker Compose's horizontal scaling feature. For example:

```
docker-compose up -d --scale edge-router=3
```

After scaling, we observed lower latency and improved throughput directly in Grafana, confirming that the additional replicas helped absorb the workload.

In a real production setup, this scaling process would be automated using tools such as Kubernetes Horizontal Pod Autoscaler or Docker Swarm autoscaling.

Automatic Self-Healing

We added basic self-healing by enabling Docker's built-in restart policies. Each microservice is configured with `restart: always`, which instructs Docker to automatically restart a container if it crashes, exits unexpectedly, or the host machine reboots.

This gives the system a simple form of resilience. If a service fails, Docker brings it back online without manual intervention.

For example, we manually killed the `docker-compose_catalogue_1` container using `docker kill`, and Docker restarted it automatically within seconds.

Summary:

Priority	Task	What We Did
High	Choose cloud technology	We selected AWS EC2 because it matched our needs and fit within the Free Tier.
High	Basic cloud deployment	We deployed the full Sock Shop microservices using Docker Compose on an EC2 instance.
High	Demo / screenshots	Demo Link: https://www.loom.com/share/a2e96f053e4642f4bbf0620445ef047c
Medium	Elasticity support	We identified high-load services in Grafana and scaled them horizontally using Docker Compose.
Medium	Automatic workload generation	We launched the built-in <code>user-sim</code> service to continuously stress-test the system.
Low	Health and performance monitoring	We deployed Prometheus, Grafana, and Alertmanager with <code>docker-compose.monitoring.yml</code> .
Low	Simple automatic autoscaling	We explored the possibility of autoscaling with docker using Prometheus with bash script.
Low	Automatic restarting of microservices	We relied on Docker's default restart behavior defined in the existing Compose files.